

Smart Wildfire Detection Dashboard

README File

Sasidhar Appalla, Praveen Krishnan Meenakshi Sundar, Nikitha Maguluri, Randell Epps

April 21, 2025

Overview

This Flask application ingests wildfire sensor readings (IR temperature, humidity, gas), classifies each reading as **LowRisk**, **MediumRisk**, or **HighRisk** using a pre-trained Random Forest, sends WhatsApp alerts on severity transitions, stores data in PostgreSQL, and serves both real-time and historical dashboards.

1 Environment Settings

Create a file named `.env` in the project root (or configure your host) with:

```
DATABASE_URL="postgres://user:password@HOST:5432/espread_data?sslmode=
    ↪ require"
TWILIO_SID="ACXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"
TWILIO_TOKEN="your_auth_token"
TWILIO_FROM="whatsapp:+14155238886"
TWILIO_TO="whatsapp:+1XXXXXXXXXX"
```

2 How to Run the Code

1. Clone & Install

```
git clone https://github.com/your-org/iotproject-flask.git
cd iotproject-flask

python3 -m venv venv
source venv/bin/activate

pip install --upgrade pip
pip install -r requirements.txt
```

2. Database Setup

Ensure the `sensor_data` table exists:

```
CREATE TABLE IF NOT EXISTS sensor_data (
  id          SERIAL PRIMARY KEY,
```

```
ts          TIMESTAMPTZ NOT NULL ,
sensors     JSONB        NOT NULL ,
location    JSONB        NOT NULL
);
```

3. Model Files

Place the following in the project root:

- `rf_model_render.pkl`
- `label_encoder_render.pkl`

4. Production with Gunicorn

```
export $(grep -v '^#' .env | xargs)
gunicorn app:app \
  -k eventlet \
  -w 1 \
  --bind 0.0.0.0:$PORT \
  --timeout 60
```

3 How to Interpret the Results

- **Summary Cards:** IR (°C), Humidity (%), Gas (MQ-135 units), Severity (green/yellow/red).
- **Map:** Latest sensor location.
- **Live Charts:** IR, Humidity, Gas over time.
- **History Table:** Last 100 readings with timestamp and severity.
- **Alerts:** One WhatsApp message when severity first transitions into MediumRisk or High-Risk.

4 API Endpoints

GET / Dashboard page.

GET /latest Returns the newest reading:

```
{
  "timestamp": 1713534809802,
  "ir": 21.63,
  "humidity": 62.7,
  "gas": 2010,
  "lat": 35.7833507,
  "lng": -78.6878767,
  "severity": "LowRisk"
}
```

GET /history Returns an array of the last 100 readings.

5 Sample Input & Output

Sensor Payload

```
{
  "timestamp": "2025-04-20T07:13:29.802414Z",
  "sensors": {"ir":21.63,"humidity":62.7,"gas":2010},
  "location":{"lat":35.7833507,"lng":-78.6878767}
}
```

/latest Response

```
{
  "timestamp": 1713534809802,
  "ir": 21.63,
  "humidity": 62.7,
  "gas": 2010,
  "lat": 35.7833507,
  "lng": -78.6878767,
  "severity": "LowRisk"
}
```

6 Additional Notes

- Adjust polling interval in `index.html` JS (`setInterval(fetchLatest, 2000)`).
- Ensure Twilio sandbox is joined to receive WhatsApp alerts.
- For deployment on Render, use the provided `render.yml`.

7 Running All Components

To fully exercise the system, you need to run three parts:

1. Flask Dashboard:

- Visit <https://iotproject-flask.onrender.com>

2. Arduino (ESP32) Sensor Node:

- Open `sensor_node.ino` in the *Arduino IDE*. Configure your Wi-Fi credentials and gateway MAC.
- Upload to your ESP32 board at 9600 baud.
- Verify Serial Monitor shows sensor readings and JSON payloads.

3. Raspberry Pi “espread” Script:

- Copy `espread5.py` to your Raspberry Pi.
- Edit the serial port (e.g. `/dev/serial0`) and `DATABASE_URL` in the script.
- `pip install -r requirements.txt` on the Raspberry Pi.

- Run `python3 espread5.py`.
- The script will read ESP32 data, optionally geolocate via Wi-Fi, publish to MQTT, and insert into Postgres.